FinPulse AWS Infrastructure & Cost Optimization Review

Date: 2026-03-16 **Status:** Validated against codebase · ready for execution **Author:** Infrastructure Review (Claude Code) **Audience:** Coding agent / engineer executing changes

---

Table of Contents

1 Executive Summary
2 Current Architecture Map
3 Resource Inventory · Verified Against Terraform
4 Cost Breakdown · Validated
5 Findings · With Code-Level Evidence
6 Dependency Chain Analysis
7 Recommendations · Phased Action Plan
8 Phase 1: Staging Cleanup (Safe, Immediate)
9 Phase 2: Redis & VPC Removal (Requires Code Changes)
10 Phase 3: Architecture Simplification (Strategic)
11 Risk Assessment & Rollback Plans
12 Verification Commands
13 Appendix: File Reference Map

---

1. Executive Summary

FinPulse is a pre-launch SaaS financial dashboard running on AWS serverless architecture, managed via Terraform. The product is operational (V3.0.0, released 2026-01-25) but has **zero active users** and **no revenue**.

Current estimated monthly cost: ~$54·60/month

The majority of this cost (~85%) comes from two infrastructure components that provide zero value at zero traffic:

- **NAT Gateway**: ~$32/month (fixed)
- **ElastiCache Redis** (prod + staging): ~$22·28/month (fixed)

Target monthly cost after optimization: ~$2·5/month

This document provides validated, code-grounded instructions to achieve this reduction in three phases.

---

2. Current Architecture Map

```
                    ...................................................
                    .                   INTERNET
.
                    ...................................................
                              .                   .
                    ...................  ...................
                    .   CloudFront    .  .   API Gateway   .
                    .  (E2Y4NTEFQ5L)  .  .  (REST, Regional).
                    .  finpulse.me    .  .  Cognito Auth    .
                    ...................  ...................
                              .                   .
                    ...................            .
```

```
                        ·   S3 Bucket     ·           ·
                        ·  (frontend SPA) ·           ·
                        ...................           ·
                                                      ·
                        ....................................................
                        ·                     VPC 10.0.0.0/16
.
                        ·  ..............................................   ·
                        ·  ·  Public Subnets (10.0.0.0/24, 10.0.1.0) ·   ·
                        ·  ·  ..................                      ·   ·
                        ·  ·  ·  NAT Gateway   · · $32/mo COST       ·   ·
                        ·  ·  ·  (1x, AZ-1)    ·                      ·   ·
                        ·  ·  ..................                      ·   ·
                        ·  ..............................................   ·
                        ·  ..............................................   ·
                        ·  ·  Private Subnets (10.0.10.0, 10.0.11.0) ·   ·
                        ·  ·                                         ·   ·
                        ·  ·  ............    ....................... ·   ·
                        ·  ·  ·  Lambda  ......  Redis (ElastiCache)· ·   ·
                        ·  ·  ·  (8 fns) ·    ·  cache.t4g.micro   · ·   ·
                        ·  ·  ·  in VPC  ·    ·  · $11-14/mo COST  · ·   ·
                        ·  ·  ............    ....................... ·   ·
                        ·  ·       ·                                 ·   ·
                        ·  ·  ..................                     ·   ·
                        ·  ·  · VPC Endpoints  ·                     ·   ·
                        ·  ·  · (DynamoDB, S3) · · FREE              ·   ·
                        ·  ·  ..................                     ·   ·
                        ·  ..............................................   ·
                        ....................................................
                                            ·
                        ........................................
                        ·  DynamoDB (12 prod + 7 staging tables) ·
                        ·  PAY_PER_REQUEST · near $0 at idle     ·
                        ........................................
    Exception: payments Lambda runs OUTSIDE VPC (no vpc_config
block)
```

---

3. Resource Inventory · Verified Against Terraform

3.1 What Terraform Actually Manages (vs. executive summary claims)

Resource	Executive Summary Claim	Terraform Reality	File Reference
NAT Gateways	2	**1** (`count = var.enable_nat_gateway ? 1 : 0`)	`networking.tf:68-87`
Elastic IPs	2	**1** (allocated to the single NAT)	`networking.tf:68-75`
Redis clusters	2 (staging + prod)	**1 replication group** (prod) + **1 cluster** (staging)	`modules/redis/main.tf:28-58`, `staging.tf:458-481`
VPCs	18 across regions	**1** (`10.0.0.0/16` in us-east-1)	`networking.tf:7-15`
Subnets	58	**4** (2 public + 2 private)	`networking.tf:38-62`
S3 buckets	3	**1 managed** (frontend), 1 state bucket (referenced)	`cloudfront.tf:13-15`
Lambda functions	"multiple"	**9 defined** (4 always-on, 5 conditional)	`modules/lambda/main.tf`

Explanation for discrepancies:

- The 18 VPCs / 58 subnets are **AWS default VPCs** auto-created in every region. They are free and harmless.
- The second NAT Gateway may be a **stale resource** from prior experimentation or a manual AWS Console creation. Verify with `aws ec2 describe-nat-gateways --filter "Name=state,Values=available"`.

3.2 Production Lambda Functions

#	Function	VPC?	Redis Used?	Memory	Timeout	File
1	`finpulse-auth-prod`	**Yes**	Rate limiting	256MB	30s	`modules/lambda/main.tf:208-260`
2	`finpulse-market-data-prod`	**Yes**	Caching (prices, FX)	256MB	30s	`modules/lambda/main.tf:264-314`
3	`finpulse-portfolio-prod`	**Yes**	Caching + rate limiting	256MB	30s	`modules/lambda/main.tf:318-368`
4	`finpulse-admin-prod`	**Yes**	Caching	256MB	30s	`modules/lambda/main.tf:535-583`
5	`finpulse-ai-prod`	**Yes**	Caching	512MB	60s	`modules/lambda/main.tf:376-425`
6	`finpulse-news-prod`	**Yes**	Caching	256MB	30s	`modules/lambda/main.tf:429-478`
7	`finpulse-community-prod`	**Yes**	Caching + rate limiting	256MB	30s	`modules/lambda/main.tf:482-531`
8	`finpulse-twitter-prod`	**Yes**	Caching	256MB	30s	`modules/lambda/main.tf:587-640`
9	`finpulse-payments-prod`	**No**	**None**	256MB	30s	`modules/lambda/main.tf:646-684`

Key observation: The `payments` Lambda already runs **outside the VPC** · it has no `vpc_config` block. This proves the pattern of running Lambda without VPC works in this codebase and is the model for all other functions.

3.3 Staging Resources (All Currently Idle)

Resource	Status	Cost	File
Staging Lambda functions	**Disabled** (`lambda_functions_staging = {}`)	$0	`staging.tf:10`
Staging Redis (`finpulse-cache-staging`)	**Running** but unused	~$11-14/mo	`staging.tf:458-481`
Staging DynamoDB (7 tables)	Running, zero reads/writes	~$0 (PAY_PER_REQUEST)	`staging.tf:188-441`
Staging Cognito user pool	Running, zero users	~$0	`staging.tf:130-182`
Staging API Gateway	Running, no integrations	~$0	`staging.tf:487-621`
Staging IAM role + policies	Exists	$0	`staging.tf:84-124`

3.4 Production DynamoDB Tables (12 Total)

All using `PAY_PER_REQUEST` billing · near-zero cost at idle.

Table	Hash Key	Range Key	GSIs	TTL	PITR
`finpulse-users-prod`	userId	·	email-index	No	Yes
`finpulse-portfolios-prod`	userId	assetId	symbol-index	No	Yes
`finpulse-market-prices-prod`	symbol	·	·	No	Yes
`finpulse-ai-queries-prod`	userId	queryId	userId-month-index	Yes	Yes
`finpulse-news-prod`	newsId	·	·	Yes	Yes
`finpulse-community-posts-prod`	postId	·	timestamp-index, userId-timestamp-index	No	Yes
`finpulse-circuit-breaker-prod`	serviceName	·	·	Yes	Yes
`finpulse-api-cache-prod`	cacheKey	·	dataType-fetchedAt-index	Yes	Yes
`finpulse-historical-prices-prod`	symbol	date	assetType-date-index	No	Yes
`finpulse-api-quota-prod`	providerDate	·	provider-index	Yes	Yes
`finpulse-subscriptions-prod`	userId	·	·	No	Yes
`finpulse-identities-prod`	userId	identityKey	email-index, provider-subject-index	No	Yes

---

4. Cost Breakdown · Validated

4.1 Current Monthly Cost Estimate

| Component | Monthly Cost | % of Total | Evidence | |----------|----------|----------|----------| | **NAT Gateway** (1x) | ~$32.40 | 54% | $0.045/hr × 720hrs = $32.40 fixed || **Redis prod** (cache.t4g.micro replication group) | ~$11.52 | 19% | $0.016/hr × 720hrs || **Redis staging** (cache.t4g.micro cluster) | ~$11.52 | 19% | $0.016/hr × 720hrs || **Elastic IP** (1x, attached) | $0 | 0% | Free when attached to running NAT | | **DynamoDB** (19 tables total) | ~$0.50 | <1% | PAY_PER_REQUEST at near-zero traffic | | **Lambda** (9 functions) | ~$0.20 | <1% | Near-zero invocations | | **API Gateway** | ~$0.10 | <1% | Near-zero requests | | **CloudFront** | ~$0.50 | <1% | Minimal bandwidth | | **S3** | ~$0.10 | <1% | Small frontend bundle | | **Cognito** | $0 | 0% | Free tier (50K MAUs) | | **CloudWatch** | ~$0.50 | <1% | 7-day retention, 10 alarms (free tier) | | **Secrets Manager** | ~$2.50 | 4% | 5 secrets × $0.40/secret + API calls || **TOTAL** | **~$59.34** | 100% | |

4.2 Cost After Full Optimization (Phase 1+2+3)

| Component | Monthly Cost | Notes | |----------|----------|-------| | Lambda | ~$0.20 | Same, outside VPC (faster cold starts) | | DynamoDB | ~$1.00 | Slight increase from rate-limit writes | | API Gateway | ~$0.10 | Same | | CloudFront | ~$0.50 | Same | | S3 | ~$0.10 | Same | | Cognito | $0 | Same | | CloudWatch | ~$0.50 | Same | | Secrets Manager | ~$2.50 | Same | | **TOTAL** | **~$4.90** | **~92% reduction** |

4.3 Cost After Phase 1 Only (Staging Cleanup)

| Removed | Savings | |---------|---------| | Staging Redis | -$11.52/mo | | **New Total** | **~$47.82/mo** |

---

5. Findings · With Code-Level Evidence

Finding 1: Staging Environment is a Ghost Town Costing Real Money

Evidence:

```
# staging.tf:4-10
locals {
  staging_suffix = "-staging"
  # Staging Lambda functions DISABLED to maximize production
concurrency
  # AWS account has 10 concurrent execution limit - need all for
production
  lambda_functions_staging = {}
}
```

Lambda functions were disabled due to a 10-concurrent-execution quota limit. However, all supporting staging infrastructure (Redis, DynamoDB, Cognito, API Gateway) remains deployed and running.

Impact: ~$11.52/month for a Redis cluster with zero connections.

Finding 2: All Lambdas (Except Payments) Are VPC-Bound

Evidence from `modules/lambda/main.tf`:

Every Lambda function (auth, market-data, portfolio, admin, ai, news, community, twitter) contains:

```
vpc_config {
  subnet_ids         = var.private_subnet_ids
  security_group_ids = [var.lambda_security_group_id]
}
```

The sole exception is `payments_service` (line 646), which has **no** `vpc_config` and runs outside the VPC.

Impact: VPC placement requires NAT Gateway ($32/mo) for internet access. It also adds ~500-1000ms cold start latency.

Finding 3: Redis Is Used for Rate Limiting and Caching, But Has Graceful Fallback

Rate limiter behavior when Redis is unavailable (`lambda-layers/shared-utils/nodejs/rate-limiter.js:59-69`):

```
// If Redis is not available, fall back to allowing the request
// (better to allow than block everyone when Redis is down)
if (!redis) {
  console.warn('[RateLimiter] Redis not available, allowing
request');
  return {
    allowed: true,
    remaining: max - 1,
    distributed: false,
  };
}
```

Cache behavior when Redis is unavailable (`lambda-layers/shared-utils/nodejs/redis-cache.js:80-84`):

```
// Skip Redis in local development if not configured
if (!process.env.REDIS_ENDPOINT && !process.env.REDIS_HOST) {
  console.log('Redis not configured, using fallback caching');
  return null;
}
```

All cache `get()` calls return `null` when Redis is unavailable (line 113: `if (!client) return null;`), and all `set()` calls return `false` (line 133: `if (!client) return false;`). The application continues to function · it simply fetches from the upstream API every time.

Impact: Removing Redis is **functionally safe** · the application degrades gracefully. However:

- Rate limiting becomes **per-instance** (each Lambda instance allows full quota) rather than distributed
- API responses are uncached (higher latency, more upstream API calls)
- At zero traffic, neither of these matters

Finding 4: NAT Gateway Is Required Only Because Lambdas Are in VPC

The dependency chain is:

```
Lambda needs internet (CoinGecko, Alpaca, etc.)
  · Lambda is in VPC (private subnet)
    · Private subnet needs NAT Gateway for internet
      · NAT Gateway costs $32/month
BUT: Lambda only needs VPC placement to reach Redis
  · If Redis is removed, VPC is unnecessary
    · If VPC is unnecessary, NAT Gateway is unnecessary
```

Proof that non-VPC Lambda works · the payments Lambda:

```
# modules/lambda/main.tf:646-684
resource "aws_lambda_function" "payments_service" {
  # ... NO vpc_config block ...
```

```
  # NOTE: Not in VPC (doesn't need Redis, only needs DynamoDB +
Secrets Manager + internet)
}
```

Finding 5: VPC Endpoints Already Exist for DynamoDB and S3

```
# networking.tf:140-160
resource "aws_vpc_endpoint" "dynamodb" {
  vpc_id            = aws_vpc.main.id
  service_name      = "com.amazonaws.${var.aws_region}.dynamodb"
  vpc_endpoint_type = "Gateway"
}
resource "aws_vpc_endpoint" "s3" {
  vpc_id            = aws_vpc.main.id
  service_name      = "com.amazonaws.${var.aws_region}.s3"
  vpc_endpoint_type = "Gateway"
}
```

These are currently used so VPC-placed Lambdas can reach DynamoDB/S3 without going through NAT. If Lambdas move out of VPC, these endpoints become unnecessary (Lambda natively reaches DynamoDB/S3 over the AWS network).

Finding 6: FX Caching Uses Multi-Tier Architecture Including Redis

From `docs/agent/MEMORY.md:100-103`:

```
Multi-tier cache: Memory (15min) · Redis (15min) · Live API · Static
fallback
Response includes source: 'live'|'redis'|'fallback' and actual
timestamp
Static rates kept as FX_FALLBACK_RATES for reliability
```

Without Redis, the FX cache chain becomes: `Memory (15min) · Live API · Static fallback`. This is adequate for pre-launch.

Finding 7: The `enable_nat_gateway` Variable Already Exists

```
# variables.tf:68-72
variable "enable_nat_gateway" {
  description = "Enable NAT Gateway (required for Lambda internet
access)"
  type        = bool
  default     = true
}
```

The NAT Gateway is already conditional. Setting `enable_nat_gateway = false` will remove it · **but only if Lambda functions are first moved out of VPC**, otherwise they lose internet access.

---

6. Dependency Chain Analysis

6.1 What Depends on What

```
Redis (ElastiCache)
  ··· Used by: rate-limiter.js (distributed sliding window)
  ··· Used by: redis-cache.js (API response caching)
  ··· Used by: cache-manager.js (multi-tier cache orchestration)
  ··· Used by: market-data Lambda (price caching)
  ··· Used by: portfolio Lambda (data caching)
  ··· Used by: community Lambda (data caching)
  ··· Used by: auth Lambda (rate limiting)
  ··· Fallback: All code gracefully handles Redis=null
VPC (Private Subnets)
  ··· Required by: Redis (ElastiCache must be in VPC)
  ··· Required by: Lambda vpc_config (8/9 functions)
  ··· NOT required by: payments Lambda (already outside VPC)
NAT Gateway
  ··· Required by: VPC-placed Lambda for internet access
  ··· NOT required if: Lambda runs outside VPC
  ··· Cost: $32.40/month (fixed, regardless of traffic)
Security Groups
  ··· lambda-sg: Allows all outbound from Lambda
  ··· redis-sg: Allows port 6379 inbound from lambda-sg
  ··· Both unnecessary if Redis and VPC are removed
```

6.2 Safe Removal Order

```
1. Remove staging Redis cluster          (no dependencies, staging
Lambdas disabled)
2. Remove all other staging resources    (no dependencies)
3. Remove REDIS_ENDPOINT from Lambda envs (triggers code fallback
path)
4. Remove vpc_config from Lambda defs    (Lambdas move to public
AWS network)
5. Remove Redis module (production)      (no longer referenced)
6. Set enable_nat_gateway = false        (no VPC Lambdas remain)
7. Remove VPC endpoints, security groups (optional cleanup)
```

Critical: Steps 3 and 4 must happen **together** in a single deployment, because:

- Without `REDIS_ENDPOINT`, Lambda doesn't try to connect to Redis (graceful fallback)
- Without `vpc_config`, Lambda runs outside VPC and gets native internet access
- If you remove `vpc_config` without removing `REDIS_ENDPOINT`, Lambda will try to connect to a Redis endpoint it can no longer reach (timeout errors)

---

7. Recommendations · Phased Action Plan

| Phase | Savings | Risk | Effort | Prerequisite | |-------|---------|------|--------|-------------| | **Phase 1:** Remove staging resources | ~$11.52/mo | None | Low (Terraform only) | None | | **Phase 2:** Remove Redis + VPC + NAT | ~$43.92/mo | Low (at zero traffic) | Medium (Terraform + Lambda env) | Phase 1 | | **Phase 3:** Simplify remaining infra | ~$0 | None | Low (cleanup) | Phase 2 | | **TOTAL** | **~$55.44/mo** | | | |

---

8. Phase 1: Staging Cleanup

Goal

Remove all staging resources that cost money or add state complexity. The staging Lambda functions are already disabled · this removes the orphaned supporting infrastructure.

Risk: NONE

- Staging Lambdas are disabled (`lambda_functions_staging = {}`)
- No traffic reaches staging resources
- No application code references staging resources

Changes Required

8.1 Terraform: Delete Staging Redis

File: `finpulse-infra/staging.tf`

Remove lines 447-481 (staging ElastiCache subnet group and cluster):

```
# DELETE THIS BLOCK:
resource "aws_elasticache_subnet_group" "staging" { ... }
# DELETE THIS BLOCK:
resource "aws_elasticache_cluster" "staging" { ... }
```

8.2 Terraform: Delete Staging DynamoDB Tables

File: `finpulse-infra/staging.tf`

Remove lines 188-441 (7 staging DynamoDB tables):

- `aws_dynamodb_table.users_staging`
- `aws_dynamodb_table.portfolios_staging`
- `aws_dynamodb_table.market_prices_staging`
- `aws_dynamodb_table.ai_queries_staging`
- `aws_dynamodb_table.news_staging`
- `aws_dynamodb_table.community_posts_staging`
- `aws_dynamodb_table.circuit_breaker_staging`

8.3 Terraform: Delete Staging Cognito

File: `finpulse-infra/staging.tf`

Remove lines 130-182:

- `aws_cognito_user_pool.staging`
- `aws_cognito_user_pool_client.staging`

8.4 Terraform: Delete Staging API Gateway

File: `finpulse-infra/staging.tf`

Remove lines 487-621 (REST API, authorizer, routes, deployment, stage):

- `aws_api_gateway_rest_api.staging`
- `aws_api_gateway_authorizer.cognito_staging`
- All staging API resources, methods, integrations
- `aws_api_gateway_deployment.staging`
- `aws_api_gateway_stage.staging`

8.5 Terraform: Delete Staging IAM

File: `finpulse-infra/staging.tf`

Remove lines 84-124:

- `aws_iam_role.lambda_exec_staging`
- `aws_iam_role_policy_attachment.lambda_basic_staging`
- `aws_iam_role_policy.lambda_secrets_staging`

8.6 Terraform: Delete Staging Lambda Definition

File: `finpulse-infra/staging.tf`

Remove lines 29-67 (the `aws_lambda_function.staging` resource with `for_each = local.lambda_functions_staging`).

8.7 Terraform: Delete Staging Outputs

File: `finpulse-infra/staging.tf`

Remove lines 627-645 (4 staging outputs):

- `staging_api_url`
- `staging_cognito_user_pool_id`
- `staging_cognito_client_id`
- `staging_lambda_functions`

8.8 Terraform: Keep Only Locals Block (For Reference)

After all deletions, `staging.tf` should contain only the locals and placeholder data source (if needed by other resources), or be deleted entirely.

Recommended approach: Delete the entire file. Keep the `data "archive_file" "placeholder"` block · move it to `modules/lambda/main.tf` if not already defined there (it is already defined there at line 165, so the staging one is a duplicate and can be removed).

8.9 Execution Steps

```
# 1. Navigate to infra directory
cd finpulse-infra
# 2. Delete or gut staging.tf (as described above)
# 3. Format
terraform fmt -recursive
# 4. Validate
terraform validate
# 5. Plan · expect only DESTROY operations for staging resources
terraform plan -out=tfplan
# 6. Review plan · should show:
#    - Destroy: 1 ElastiCache cluster
#    - Destroy: 1 ElastiCache subnet group
#    - Destroy: 7 DynamoDB tables
#    - Destroy: 1 Cognito user pool + client
#    - Destroy: 1 API Gateway REST API + resources
#    - Destroy: 1 IAM role + policies
#    - Destroy: Lambda function resource (empty for_each = no
actual Lambdas)
```

```
#    - NO changes to production resources
terraform show tfplan
# 7. Apply (REQUIRES EXPLICIT USER APPROVAL · Level C action)
terraform apply tfplan
```

---

9. Phase 2: Redis & VPC Removal

Goal

Remove production Redis, move all Lambda functions out of VPC, and disable NAT Gateway. This eliminates ~$43.92/month.

Risk: LOW at zero traffic

- Rate limiting degrades to per-instance (acceptable with zero users)
- Caching is lost (acceptable · upstream APIs handle the load)
- Lambda cold starts improve by ~500-1000ms (VPC ENI creation eliminated)

Prerequisites

- Phase 1 completed
- Confirm zero active users (check CloudWatch Lambda invocation metrics)

Changes Required

9.1 Lambda Environment Variables: Remove REDIS_ENDPOINT

File: `finpulse-infra/modules/lambda/main.tf`

For each Lambda function (`auth_service`, `market_data_service`, `portfolio_service`, `admin_service`, `ai_service`, `news_service`, `community_service`, `twitter_service`), remove the `REDIS_ENDPOINT` line from the `environment.variables` block.

Example for auth_service (line 239-246):

```
# BEFORE:
environment {
  variables = {
    ENVIRONMENT       = var.environment
    COGNITO_POOL_ID   = var.cognito_user_pool_id
    COGNITO_CLIENT_ID = var.cognito_client_id
    REDIS_ENDPOINT    = var.redis_endpoint    # · REMOVE THIS LINE
    ALLOWED_ORIGIN    = var.allowed_origin
  }
}
# AFTER:
environment {
  variables = {
    ENVIRONMENT       = var.environment
    COGNITO_POOL_ID   = var.cognito_user_pool_id
    COGNITO_CLIENT_ID = var.cognito_client_id
    ALLOWED_ORIGIN    = var.allowed_origin
```

```
  }
}
```

Repeat for all 8 VPC-placed Lambda functions.

Important: Because all Lambda functions have `lifecycle { ignore_changes = [environment] }`, Terraform will NOT update the environment variables. You must **also** update them via AWS CLI or temporarily remove the `ignore_changes` for `environment`. Options:

Option A (recommended): Use AWS CLI to update env vars after Terraform changes:

```
# For each function, get current config and remove REDIS_ENDPOINT:
for fn in auth market-data portfolio admin news community twitter;
do
  aws lambda update-function-configuration \
    --function-name "finpulse-${fn}-prod" \
    --environment
"Variables={ENVIRONMENT=prod,ALLOWED_ORIGIN=https://finpulse.me}" \
    --region us-east-1
done
```

Note: The exact variables differ per function · use `aws lambda get-function-configuration --function-name finpulse-auth-prod` to get current values, then reconstruct without `REDIS_ENDPOINT`.

Option B: Temporarily remove `environment` from `ignore_changes` in each Lambda's `lifecycle` block, run `terraform apply`, then add it back.

9.2 Lambda: Remove vpc_config Blocks

File: `finpulse-infra/modules/lambda/main.tf`

Remove the `vpc_config` block from all 8 Lambda functions. Example:

```
# DELETE THIS BLOCK from each Lambda:
vpc_config {
  subnet_ids         = var.private_subnet_ids
  security_group_ids = [var.lambda_security_group_id]
}
```

Functions to modify:

- `aws_lambda_function.auth_service` (line 234-237)
- `aws_lambda_function.market_data_service` (line 290-293)
- `aws_lambda_function.portfolio_service` (line 346-349)
- `aws_lambda_function.admin_service` (line 557-560)
- `aws_lambda_function.ai_service` (line 401-404)
- `aws_lambda_function.news_service` (line 456-459)
- `aws_lambda_function.community_service` (line 508-511)
- `aws_lambda_function.twitter_service` (line 614-617)

Note: The `payments_service` (line 646) already has no `vpc_config` · do not modify it.

9.3 Lambda Module: Remove VPC-Related Variables

File: `finpulse-infra/modules/lambda/variables.tf`

Remove or mark as optional:

- `variable "private_subnet_ids"` (line 31-34)
- `variable "lambda_security_group_id"` (line 36-39)
- `variable "redis_endpoint"` (line 47-50)

File: `finpulse-infra/infrastructure.tf`

Remove from the `module "lambda"` block (line 79-110):

- `private_subnet_ids` (line 87)
- `lambda_security_group_id` (line 88)
- `redis_endpoint` (line 91)

Remove:

- `depends_on = [module.redis, module.secrets]` · change to `depends_on = [module.secrets]`

9.4 Lambda Module: Remove VPC IAM Policy

File: `finpulse-infra/modules/lambda/main.tf`

Remove the VPC access policy (lines 59-84):

```
# DELETE THIS ENTIRE RESOURCE:
resource "aws_iam_role_policy" "lambda_vpc" { ... }
```

9.5 Infrastructure: Remove Redis Module

File: `finpulse-infra/infrastructure.tf`

Remove lines 40-52:

```
# DELETE THIS BLOCK:
module "redis" {
  source = "./modules/redis"
  ...
}
```

9.6 CloudWatch: Remove Redis Alarm

File: `finpulse-infra/infrastructure.tf`

Remove from `module "cloudwatch"` block:

- `redis_cluster_id = module.redis.cluster_id` (line 150)
- `redis_memory_threshold = var.redis_memory_threshold` (line 151)

File: `finpulse-infra/modules/cloudwatch/main.tf` and `variables.tf`

Remove the Redis memory alarm resource and associated variables.

9.7 Networking: Disable NAT Gateway

File: `finpulse-infra/variables.tf`

Change the default:

```
variable "enable_nat_gateway" {
  description = "Enable NAT Gateway (required for Lambda internet
access)"
```

```
  type        = bool
  default     = false   # · Changed from true
}
```

Or set in `terraform.tfvars`:

```
enable_nat_gateway = false
```

9.8 Networking: Remove Security Groups (Optional Cleanup)

File: `finpulse-infra/networking.tf`

Remove or comment out:

- `aws_security_group.lambda` (lines 167-183) · no longer needed without VPC Lambdas
- `aws_security_group.redis` (lines 186-210) · no longer needed without Redis

9.9 Networking: Remove VPC Endpoints (Optional Cleanup)

File: `finpulse-infra/networking.tf`

Remove:

- `aws_vpc_endpoint.dynamodb` (lines 140-149) · Lambda reaches DynamoDB natively outside VPC
- `aws_vpc_endpoint.s3` (lines 151-160) · same

9.10 Outputs: Remove Redis Output

File: `finpulse-infra/outputs.tf`

Remove lines 76-82:

```
# DELETE THIS BLOCK:
output "redis" {
  description = "Redis connection info"
  value = {
    endpoint = module.redis.endpoint
    port     = module.redis.port
  }
}
```

9.11 Execution Steps

```
cd finpulse-infra
# 1. Make all Terraform changes described above
# 2. Format and validate
terraform fmt -recursive
terraform validate
# 3. Plan · expect:
#    - Destroy: 1 ElastiCache replication group
#    - Destroy: 1 ElastiCache subnet group (production)
#    - Destroy: 1 NAT Gateway
#    - Destroy: 1 Elastic IP
#    - Destroy: 2 VPC endpoints (optional)
#    - Destroy: 2 security groups (optional)
#    - Update: 8 Lambda functions (remove vpc_config)
```

```
#    - NO changes to DynamoDB, Cognito, API Gateway, CloudFront, S3
terraform plan -out=tfplan
terraform show tfplan
# 4. Apply (REQUIRES EXPLICIT USER APPROVAL · Level C action)
terraform apply tfplan
# 5. Update Lambda environment variables via CLI
#    (because lifecycle ignore_changes prevents Terraform from
updating them)
#    See step 9.1 Option A above
```

---

10. Phase 3: Architecture Simplification

Goal

Clean up remaining infrastructure complexity after Phases 1 and 2.

10.1 Consider Removing the VPC Entirely

After Phases 1 and 2, the VPC is empty · no Lambda functions, no Redis, no resources in private subnets. The VPC, subnets, route tables, and IGW cost nothing, but they add Terraform state complexity.

Options:

- **Keep the VPC** (zero cost, useful if Redis is reintroduced later)
- **Remove the VPC** (cleaner state, but requires re-creation if needed)

Recommendation: Keep the VPC shell. It costs nothing and allows fast reintroduction of VPC resources.

10.2 Review Variables for Dead Defaults

After Phase 2, these variables become unused:

- `redis_node_type` · remove
- `redis_num_cache_nodes` · remove
- `redis_memory_threshold` · remove
- `enable_api_caching` · keep (API Gateway caching is independent of Redis)

10.3 Final Target Architecture

```
..............    ...............    ................
· CloudFront ......  S3 Bucket  ·    ·   Cognito    ·
·            ·    · (frontend)  ·    · (user pool)  ·
..............    ...............    ................
                                            .
                  ...............           .
                  · API Gateway .............
                  · (REST API)  · (JWT authorizer)
                  ...............
                         .
               .......................
               ·   Lambda Functions  ·
               ·   (outside VPC)     ·
               ·   9 microservices   ·
```

```
                    .......................
                              .
                    .......................
                    ·      DynamoDB       ·
                    ·   (12 tables,       ·
                    ·    PAY_PER_REQUEST) ·
                    .......................
Monthly cost: ~$2-5
```

10.4 When to Reintroduce Removed Components

Trigger	Action
> 100 DAU	Consider API Gateway caching ($11/mo)
> 1,000 DAU	Reintroduce Redis for rate limiting + caching
Compliance requirement	Reintroduce VPC for network isolation
> 10,000 DAU	Add NAT Gateway + multi-AZ Redis

---

11. Risk Assessment & Rollback Plans

Phase 1 Rollback

Risk: None · staging resources are unused**Rollback:** Re-run `terraform apply` with the original `staging.tf` file from git.

Phase 2 Rollback

Risk: Low (only relevant if traffic appears during migration).

Scenario	Impact	Mitigation
Lambda loses internet	External API calls fail (CoinGecko, Alpaca)	Verify NAT removal happens AFTER vpc_config removal
Rate limiting disabled	All requests allowed (open throttle)	Acceptable at zero traffic; add API Gateway throttling as backup
Cache miss on every request	Higher latency, more upstream API calls	Acceptable at zero traffic; upstream APIs have generous free tiers
Need to revert	Takes ~15 minutes	`git revert` the commit + `terraform apply`

Rollback procedure:

```
# 1. Revert the commit
git revert HEAD
# 2. Re-apply original Terraform
cd finpulse-infra
terraform plan -out=tfplan
terraform apply tfplan
# 3. Restore Lambda environment variables
# (if using Option A from 9.1, re-add REDIS_ENDPOINT via CLI)
```

---

12. Verification Commands

Pre-Change Verification

```
# Confirm AWS identity
aws sts get-caller-identity
# Confirm Terraform state
cd finpulse-infra
```

```
terraform version
terraform workspace show
# Check current NAT Gateways
aws ec2 describe-nat-gateways --filter
"Name=state,Values=available" \
  --query
"NatGateways[].{Id:NatGatewayId,SubnetId:SubnetId,State:State}" \
  --output table
# Check current ElastiCache clusters
aws elasticache describe-cache-clusters \
  --query

"CacheClusters[].{Id:CacheClusterId,Engine:Engine,NodeType:CacheNodeType,Status:Cache
  --output table
# Check current Lambda VPC configs
for fn in auth market-data portfolio admin news community twitter
payments; do
  echo "=== finpulse-${fn}-prod ==="
  aws lambda get-function-configuration \
    --function-name "finpulse-${fn}-prod" \
    --query
"{VpcConfig:VpcConfig.VpcId,SubnetIds:VpcConfig.SubnetIds}" \
    --output table 2>/dev/null || echo "Not found"
done
# Check Lambda invocation count (last 7 days)
aws cloudwatch get-metric-statistics \
  --namespace AWS/Lambda \
  --metric-name Invocations \
  --dimensions Name=FunctionName,Value=finpulse-auth-prod \
  --start-time $(date -u -d '7 days ago' +%Y-%m-%dT%H:%M:%S) \
  --end-time $(date -u +%Y-%m-%dT%H:%M:%S) \
  --period 604800 \
  --statistics Sum \
  --output table
```

Post-Change Verification (Phase 1)

```
# Confirm staging Redis is gone
aws elasticache describe-cache-clusters \
  --query
"CacheClusters[?CacheClusterId=='finpulse-cache-staging']" \
  --output table
# Expected: empty result
# Confirm staging DynamoDB tables are gone
aws dynamodb list-tables --query "TableNames[?contains(@,
'staging')]" --output table
# Expected: empty result
# Confirm production resources are untouched
aws elasticache describe-replication-groups \
  --query
"ReplicationGroups[?ReplicationGroupId=='finpulse-redis-prod']" \
  --output table
# Expected: still running
```

Post-Change Verification (Phase 2)

```
# Confirm Redis is gone
aws elasticache describe-replication-groups \
  --query "ReplicationGroups[?contains(ReplicationGroupId,
'finpulse')]" \
  --output table
# Expected: empty result
# Confirm NAT Gateway is gone
aws ec2 describe-nat-gateways --filter
"Name=state,Values=available" \
  --query "NatGateways[].NatGatewayId" --output table
# Expected: empty or no finpulse NAT
# Confirm Lambda functions are outside VPC
for fn in auth market-data portfolio admin news community twitter
payments; do
  VPC=$(aws lambda get-function-configuration \
    --function-name "finpulse-${fn}-prod" \
    --query "VpcConfig.VpcId" --output text 2>/dev/null)
  echo "finpulse-${fn}-prod: VPC=${VPC:-NONE}"
done
# Expected: all show VPC=NONE or VPC=
# Test a Lambda function end-to-end
curl -s https://YOUR_API_GATEWAY_URL/market/prices?type=crypto |
head -c 200
# Expected: valid JSON response
```

---

13. Appendix: File Reference Map

Terraform Files · What To Change

File	Phase	Action	Lines Affected
`finpulse-infra/staging.tf`	1	**Delete entire file**	All 646 lines
`finpulse-infra/modules/lambda/main.tf`	2	Remove `vpc_config` blocks (8 functions), remove `lambda_vpc` IAM policy	~30 lines removed
`finpulse-infra/modules/lambda/variables.tf`	2	Remove `private_subnet_ids`, `lambda_security_group_id`, `redis_endpoint`	~12 lines removed
`finpulse-infra/infrastructure.tf`	2	Remove `module "redis"`, update `module "lambda"` inputs, update `module "cloudwatch"`	~20 lines changed
`finpulse-infra/variables.tf`	2	Change `enable_nat_gateway` default to `false`, remove redis variables	~10 lines changed
`finpulse-infra/networking.tf`	2 (optional)	Remove security groups, VPC endpoints	~70 lines removed
`finpulse-infra/outputs.tf`	2	Remove `redis` output	7 lines removed

Application Code · No Changes Required

The application code (`lambda-code/`, `lambda-layers/`) does **not** need modification. The Redis fallback paths are already implemented:

File	Behavior When Redis Is Absent
`lambda-layers/shared-utils/nodejs/redis-cache.js`	Returns `null` for gets, `false` for sets · caller fetches from upstream
`lambda-layers/shared-utils/nodejs/rate-limiter.js`	Allows all requests with `distributed: false` warning

`lambda-layers/shared-utils/nodejs/cache-manager.js` | Falls through to next cache tier | |
`lambda-code/market-data/index.js` | Fetches from CoinGecko/Alpaca directly | |
`lambda-code/portfolio/index.js` | Fetches from DynamoDB directly | |
`lambda-code/community/index.js` | Fetches from DynamoDB directly | |
`lambda-code/auth/index.js` | Rate limiting degrades to allow-all |

Key Architecture Decision Records

| ADR | Relevance | |-----|-----------| | ADR-009 (Lambda Layer) | Layer deployment is independent of Redis · it will continue to work | | ADR-013 (JWT Decode Fallback) | JWT verification is independent of Redis | | ADR-011 (Production Hardening) | Rate limiting was added as security hardening · document its temporary removal |

Document History

| Date | Version | Change | |------|---------|--------| | 2026-03-16 | 1.0 | Initial validated review against codebase |

End of document. All findings verified against FinPulse repository commit history and Terraform source code.